

Hospital Patient Care Analytics Pipeline

An End-to-End ETL Data Engineering Solution

Case Study – Data Engineering Fundamentals

Submitted by: METLA KUMARA SWAMY

Roll Number: 2300031747

Tools: Python, Pandas, SQLAlchemy, MySQL, Matplotlib, Streamlit

1. Problem Statement

A multi-specialty hospital collects data from several separate systems: patient registration, appointment scheduling, laboratory reports, wearable health devices, and doctor consultation notes. Because this data sits in different systems and formats, management cannot easily see where delays happen or which patients need attention.

The hospital wants a data-driven solution that can **collect, process, store, and analyse** patient data to improve operational efficiency and patient outcomes.

1.1 Objectives

The pipeline is designed around three business goals:

Business goal	What the pipeline delivers
Reduce waiting times	Waiting time per visit, average wait by department and by hour of day, no-show rates
Improve patient care	One clean, unified view of each patient: profile, visits, lab results, vitals and notes
Predict high-risk patients	A risk score (Low / Medium / High) for every patient built from age, chronic disease, abnormal labs, abnormal vitals and critical symptoms

1.2 Scope

This project implements a complete batch **ETL (Extract, Transform, Load)** pipeline in Python that loads clean data into a MySQL database, followed by SQL analysis, dashboard charts, a clickable web application, and an emergency appointment scheduler that uses the loaded data to prioritise urgent patients when a doctor is fully booked. Advanced components such as a data lake, cloud data warehouse and real-time streaming are described in Section 15 (Future Scope).

2. Understanding the Data Sources

Each hospital system produces data of a different type and format. Recognising this **variety** is the first step in designing the pipeline, because each source needs its own extraction and cleaning logic.

Source system	File / format	Data type	Main problems found
Patient registration	patients.csv	Structured	Duplicate registrations, missing city / blood group, gender written 6 different ways, extra spaces in names
Appointment scheduling	appointments.csv + upcoming_schedule.csv	Structured (transactional)	Consultation time before check-in (entry errors), appointments for non-existent patients
Laboratory reports	lab_reports.json	Semi-structured (nested JSON)	Numbers stored as text, missing results
Wearable devices	wearables.csv	Time-series (high volume)	Sensor errors (heart rate 0 or 300), missing readings, duplicate transmissions

Source system	File / format	Data type	Main problems found
Doctor consultation notes	doctor_notes.csv	Unstructured (free text)	Inconsistent case (some notes in CAPITALS), information hidden inside sentences

The scheduling system provides two exports: **appointments.csv** (past visits, used to analyse waiting times) and **upcoming_schedule.csv** (tomorrow's booked slots for every doctor, used by the emergency scheduler).

Why sample data? Real patient records are protected by privacy law and cannot be used in a student project. The script `generate_data.py` therefore creates realistic sample data for all five systems. It deliberately includes the problems listed above so that the Transform step performs genuine cleaning, as it would on real hospital data. A fixed random seed makes the data identical on every run, so results are reproducible.

3. Solution Architecture and Workflow

The solution follows the classic ETL pattern with an added validation gate before loading. The workflow below shows how data moves from the five source systems to the final database, analysis and dashboards.

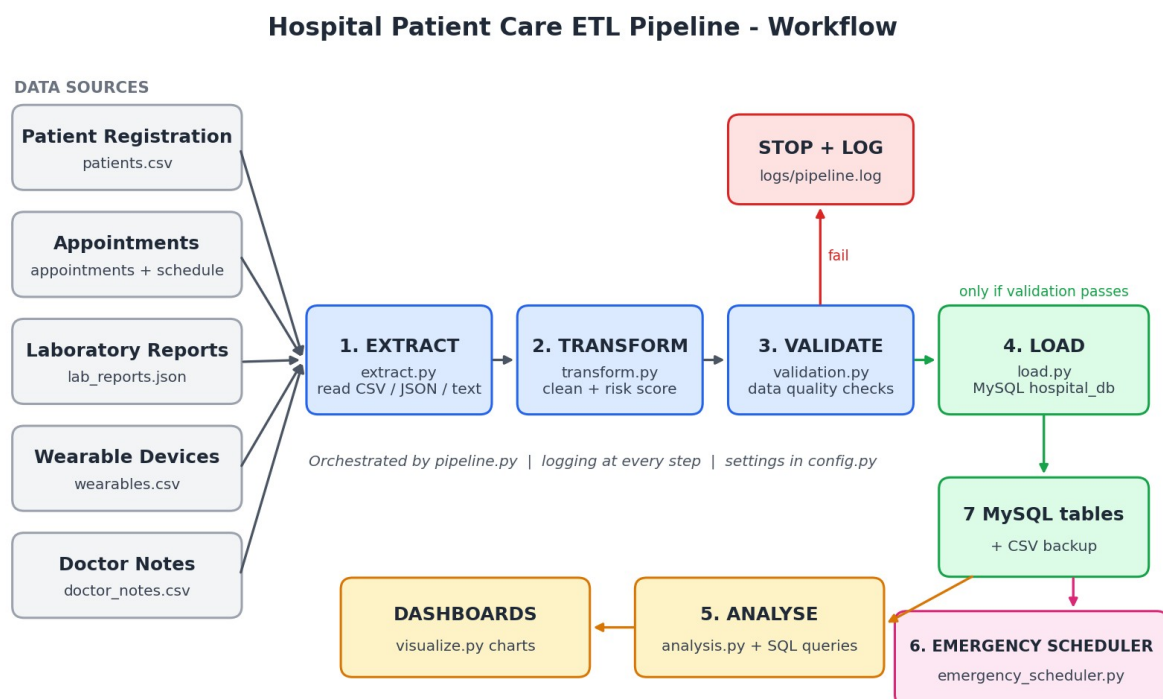


Figure 1: End-to-end workflow of the Hospital ETL pipeline

3.1 Workflow Steps

1. **Data sources:** five hospital systems export CSV, JSON and text files into `data/raw/`.
2. **Extract:** each source is read with the correct reader and converted into a Pandas DataFrame.
3. **Transform:** data is cleaned, standardised and enriched with new columns; all sources are then combined into a patient risk table.

4. **Validate:** data quality rules are checked. If any rule fails, the pipeline stops and logs the error, so bad data never reaches the database.
5. **Load:** seven clean tables are written to the MySQL database `hospital_db`, with a CSV backup in `data/processed/`.
6. **Analyse:** SQL queries answer the business questions, and `visualize.py` produces dashboard charts.
7. **Emergency scheduling:** `emergency_scheduler.py` uses the loaded schedule and risk scores to find a slot for an emergency patient when the doctor is fully booked.

The whole flow is orchestrated by **pipeline.py**, which runs each step in order, logs progress to `logs/pipeline.log`, and reads all settings from **config.py**.

4. Project Structure

The project follows the same modular structure used in the classroom ETL exercise: one Python file per pipeline stage, tied together by `pipeline.py`. This keeps each stage easy to read, test and change independently.

```
hospital_etl/
├── config.py           # DB settings, paths, business rules
├── generate_data.py    # creates sample source data
├── extract.py          # E - read all 5 sources
├── transform.py        # T - clean, enrich, risk score
├── validation.py       # data quality checks
├── load.py             # L - load into MySQL + CSV backup
├── pipeline.py         # runs the full pipeline
├── analysis.py         # runs business SQL queries
├── visualize.py        # dashboard charts
├── emergency_scheduler.py # emergency appointment prioritisation + booking
├── app.py              # clickable web app (Streamlit)
├── sql/analysis_queries.sql
├── data/raw/           # source files
├── data/processed/     # clean CSV output
├── reports/            # chart images
├── logs/pipeline.log
├── docs/workflow_diagram.png
├── requirements.txt
└── README.md
```

5. Step 1 – Extract (`extract.py`)

The Extract step reads data from each source system **without changing it**. Because the sources use different formats, each has its own function:

Function	Source	How it is read
<code>extract_patients()</code>	Registration	<code>pd.read_csv()</code>
<code>extract_appointments()</code>	Scheduling	<code>pd.read_csv()</code>
<code>extract_lab_reports()</code>	Laboratory	<code>json.load()</code> + <code>pd.json_normalize()</code> to flatten nested JSON into a table
<code>extract_wearables()</code>	Wearable devices	<code>pd.read_csv()</code> on the exported device feed
<code>extract_doctor_notes()</code>	Consultation notes	<code>pd.read_csv()</code> with free-text column
<code>extract_upcoming_schedule()</code>	Scheduling	<code>pd.read_csv()</code>

Function	Source	How it is read
)	(tomorrow)	

The main `extract()` function calls all five and returns a dictionary of six raw DataFrames, logging the row and column count of each.

```
def extract_lab_reports():
    with open(SOURCE_FILES["lab_reports"]) as f:
        data = json.load(f)
    return pd.json_normalize(data["reports"]) # flatten JSON into a table
```

Result: 206 patient rows, 1,200 appointments, 713 lab reports, 4,360 wearable readings, 450 doctor notes and 140 upcoming appointment slots were extracted.

6. Step 2 – Transform (transform.py)

The Transform step is where most of the work happens. Each table is cleaned (fixing problems) and enriched (adding useful new columns).

6.1 Cleaning and Enrichment per Source

Table	Cleaning performed	New columns created
patients	Removed 6 duplicates; trimmed and title-cased names; standardised gender (M / male / Male → Male); filled missing city and blood group with 'Unknown'	age, age_group
appointments	Removed 5 rows for unknown patients (referential integrity); removed 8 rows where consultation was before check-in	waiting_minutes, appointment_hour, day_of_week, is_no_show
lab_results	Converted text numbers to numeric; removed 22 missing results	ref_low, ref_high, result_status (Low / Normal / High), is_abnormal
vitals	Removed 40 duplicate transmissions and 60 sensor-error readings outside physically possible ranges	is_abnormal_vital
doctor_notes	Converted text to lowercase and trimmed spaces	symptoms_found, has_critical_symptom (simple keyword-based text extraction)
upcoming_appointments	Converted times to datetime; triage level to whole numbers; bookings for unknown patients freed	slot_status (Booked / Free)

Key derived metric, **waiting time**, is calculated as:

```
waiting_minutes = consultation_start_time - checkin_time (in minutes)
```

6.2 Patient Risk Score

All five sources are combined into one row per patient (the `patient_risk_scores` table). A transparent, rule-based score is then calculated. Each risk factor adds points:

Risk factor	Source	Points
Age 60 or above	Registration	+2
Has a chronic condition (diabetes, heart disease, etc.)	Registration	+2
Number of abnormal lab results	Laboratory	+1 each (max 3)
More than 30% of wearable readings abnormal	Wearables	+2
Notes mentioning chest pain, shortness of breath or palpitations	Doctor notes	+1 each (max 2)

Risk level: **Low** = 0–3 points, **Medium** = 4–6 points, **High** = 7 or more points. A rule-based score was chosen because it is easy for doctors to understand and verify. Section 15 explains how it can later be replaced by a machine learning model.

Result: 135 patients Low risk, 58 Medium risk, 7 High risk.

7. Step 3 – Validate (validation.py)

Before loading, the clean data must pass data quality checks written with assert statements, the same approach used in the classroom exercise. If any check fails, an AssertionError stops the pipeline and the error is logged, so incorrect data never enters the database.

Table	Checks
patients	patient_id unique; no null values; gender only Male/Female; age between 0 and 120
appointments	appointment_id unique; every patient_id exists; valid status; completed visits have a non-negative waiting time
lab_results	No missing results; result_status only Low / Normal / High
vitals	Heart rate, SpO2 and temperature within valid ranges; no duplicate readings
doctor_notes	No empty notes
patient_risk_scores	Exactly one row per patient; valid risk level
upcoming_appointments	No doctor booked twice at the same time; valid slot type and triage level; booked slots belong to real patients

Result: all checks passed.

8. Step 4 – Load (load.py)

The Load step writes each clean table into the MySQL database **hospital_db** using SQLAlchemy and Pandas to_sql(), following the same method used in class. Improvements over the classroom version:

- The database is created automatically if it does not exist (CREATE DATABASE IF NOT EXISTS).
- Seven tables are loaded in one run instead of a single table.
- A CSV copy of every clean table is saved in data/processed/ as a backup.
- The password is read from an environment variable instead of being written in config.py.

MySQL table	Rows loaded	Description
patients	200	Clean patient master data
appointments	1,187	Visits with waiting time and no-show flag
lab_results	691	Lab tests with Low / Normal / High status
vitals	4,260	Valid wearable readings with abnormal flag
doctor_notes	450	Notes with extracted symptoms
patient_risk_scores	200	One row per patient with risk score and level
upcoming_appointments	140	Tomorrow's schedule: 10 doctors × 14 slots

8.1 Pipeline Execution Log

Output of running python pipeline.py (also saved in logs/pipeline.log):

```

INFO | STEP 1/4: EXTRACT
INFO | Extracted patients      :   206 rows, 8 columns
INFO | Extracted appointments : 1200 rows, 8 columns
INFO | STEP 2/4: TRANSFORM
INFO | patients      : removed 6 duplicates
INFO | appointments : removed 5 unknown-patient rows, 8 invalid times
INFO | wearables     : removed 40 duplicates, 60 sensor errors
INFO | schedule      : 121 of 140 slots booked for 2026-09-22
INFO | patient_risk : {'Low': 135, 'Medium': 58, 'High': 7}
INFO | STEP 3/4: VALIDATE
INFO | Validation Passed - all data quality checks OK
INFO | STEP 4/4: LOAD
INFO | Loaded 1187 rows into 'appointments' table
INFO | ===== Pipeline finished successfully =====

```

9. Step 5 – Analysis and Insights

After loading, analysis.py runs the SQL queries stored in sql/analysis_queries.sql and visualize.py creates dashboard charts. The results below answer the hospital's three business goals.

9.1 Goal: Reduce Waiting Times

Department	Completed visits	Avg wait (min)	Max wait (min)
General Medicine	161	48.3	91
Cardiology	155	43.6	85
Orthopedics	172	39.5	78
Pulmonology	174	32.8	76
Neurology	155	32.3	66
Pediatrics	166	28.1	65

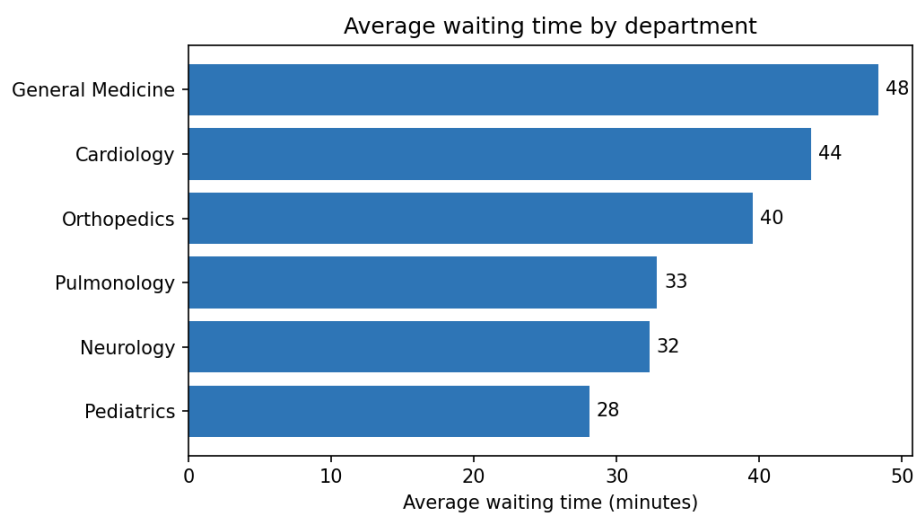


Figure 2: Average waiting time by department

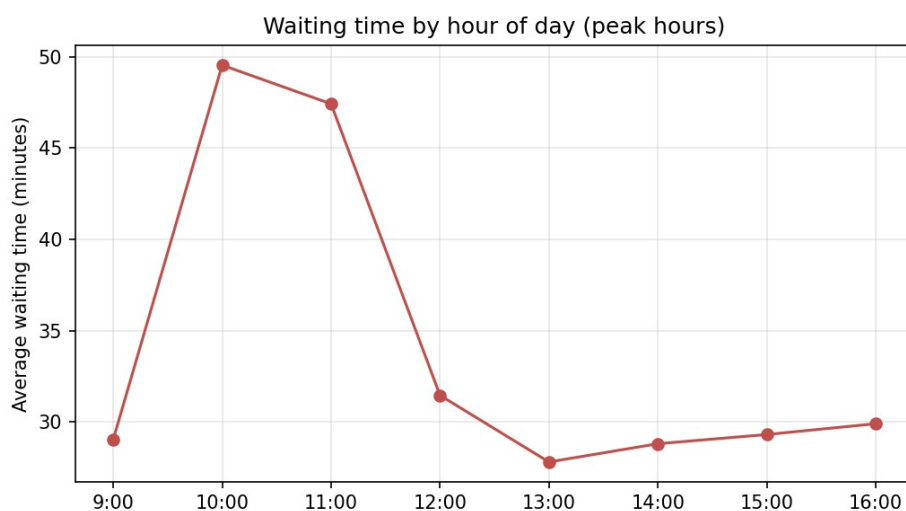


Figure 3: Average waiting time by hour of day

Insights: General Medicine has the longest average wait (48 minutes). Waiting time peaks sharply at **10:00–11:00 AM** (about 48–50 minutes), compared with about 28–31 minutes at other times.

Recommendation: add doctors or spread appointment slots during the 10–12 AM rush, especially in General Medicine and Cardiology. Pediatrics has the highest no-show rate (17.2%), so SMS reminders could free up slots.

9.2 Goal: Predict High-Risk Patients

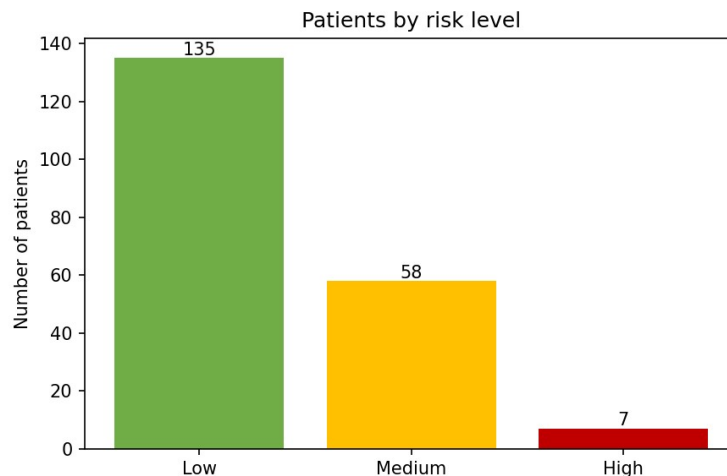


Figure 4: Patients by risk level

Patient	Age	Condition	Abnormal labs	Abnormal vitals	Critical notes	Score
P0075	62	Hypertension	2	59%	5	10
P0098	62	Heart Disease	2	47%	2	10
P0014	68	Heart Disease	1	50%	3	9
P0146	77	Heart Disease	1	0%	2	7
P0101	75	Heart Disease	1	0%	3	7

Insight: 7 patients are classified as High risk. All are aged 60+ with heart disease or hypertension, and most have abnormal vitals from wearables and critical symptoms in doctor notes. Combining data from all five sources identifies them in a way no single system could.

9.3 Goal: Improve Patient Care

The most frequently abnormal lab tests are **Total Cholesterol** (25.5% of tests abnormal) and **Fasting Blood Sugar** (23.4%), indicating that cardiac and diabetic care programmes would benefit the largest number of patients. Doctors can also view a single unified record per patient instead of checking five systems.

10. Emergency Appointment Prioritisation

10.1 The Problem

A patient needs an urgent appointment with a doctor, but every slot that day is already booked. The hospital must see the emergency patient quickly while disturbing other patients as little as possible. `emergency_scheduler.py` solves this using the data loaded by the pipeline.

An important distinction: a **life-threatening** emergency (for example heart attack symptoms or severe breathing difficulty) goes straight to the Emergency Department, which works without appointments. The scheduler is for **urgent but not life-threatening** cases that need a specific specialist today.

10.2 Data Used

- **upcoming_appointments:** tomorrow's schedule. Each of the 10 doctors has 14 slots of 30 minutes. The **10:30 slot is kept as an emergency buffer**; 10:30 was chosen because the waiting-time analysis (Section 9.1) showed 10–11 AM is the busiest period.
- **patient_risk_scores:** the risk score of the emergency patient, and the no-show history of every booked patient.

Doctor	Department	Booked slots	Buffer free?
D01	Cardiology	14 / 14	No
D02	Cardiology	14 / 14 (urgent follow-ups)	No
D03 – D05	General Medicine	10 – 12 / 14	Yes
D06	Orthopedics	14 / 14	No
D07	Orthopedics	11 / 14	Yes
D09	Neurology (only doctor)	14 / 14	No

10.3 Triage and Priority Score

A nurse first assigns a **triage level** based on how urgent the case is. The scheduler then combines it with the patient's risk score from the pipeline:

```
priority = (5 - triage_level) x 10 + risk_score
```

Triage level	Meaning	Base points	Example
1	Immediate (life-threatening)	40	Chest pain with collapse → Emergency Department
2	Very urgent	30	Heart patient with worsening symptoms
3	Urgent	20	Diabetic patient with very high sugar
4	Routine	10	Regular check-up (all normal bookings)

Triage is multiplied by 10 so it always matters most; the risk score breaks ties between patients with the same triage level. For example, a very urgent (triage 2) patient with risk score 10 gets priority 40, while a routine booking with risk score 2 has priority 12.

10.4 Decision Steps

The scheduler tries the following options in order, starting with the one that disturbs other patients least:

Step	Action	Used when
0	Send to Emergency Department	Triage level 1 (life-threatening)
1	Book the emergency buffer slot	The doctor's 10:30 buffer slot is still free
2	Double-book a likely no-show slot	A booked patient has missed 40% or more of past visits (at least 3 visits). Both patients are kept; the regular patient is phoned to confirm
3	Book another doctor in the same department	The requested doctor is full but a colleague has a free slot
4	Reschedule the lowest-priority patient	A booked patient has lower priority than the emergency; they are moved to the next day and informed by SMS
5	Overbook an extra slot at 17:00	Last resort: every booked patient has equal or higher priority

10.5 Results of the Demo Scenarios

Running `python emergency_scheduler.py` tests six scenarios, one for each step. Decisions are also saved to `reports/emergency_decisions.csv`.

Scenario	Patient (priority)	Decision
Heart attack symptoms, D01	P0101 (47)	Step 0: Emergency Department immediately
General Medicine, D03	P0149 (35)	Step 1: D03 buffer slot at 10:30
Cardiology D01 full	P0098 (40)	Step 2: double-booked at 14:30 with P0034, who missed 2 of 5 past visits
Orthopedics D06 full	P0024 (34)	Step 3: moved to D07 at 09:30
Neurology D09 full, only doctor	P0145 (36)	Step 4: takes 16:30; routine low-risk patient P0100 (priority 10) rescheduled
Cardiology D02 full of urgent follow-ups	P0057 (25)	Step 5: overbooked at 17:00, since all booked patients have priority 30+

Sample output for one scenario:

```
SCENARIO0: Neurology fully booked (only one doctor)
Patient   : P0145 Kavya Reddy, age 90, risk Medium
Triage    : 2 (Very urgent) -> priority score 36
Checked   : buffer slot already used | no likely no-show patients |
           : no free Neurology doctor
DECISION  : Step 4 - RESCHEDULE LOWEST-PRIORITY PATIENT
           : Doctor D09 at 16:30
Why       : P0100 (routine, risk Low, priority 10) moved to next
           : available day and notified by SMS.
```

A single case can also be scheduled directly, for example: `python emergency_scheduler.py --patient P0098 --doctor D01 --triage 2`

10.6 Routine Patients

Emergency options (buffer slot, double-booking, rescheduling others, overbooking) are only for triage levels 2 and 3. A **routine** request (triage 4) never disturbs other patients: it gets the next free regular slot with the doctor, or is booked for the next day if none is free.

10.7 Saving the Booking

After a decision is confirmed, the booking is **written back to MySQL**, so the next request sees the updated schedule:

Decision	Change made in upcoming_appointments
Step 1 / 3 – free slot	The free slot is filled with the emergency patient
Step 2 – double-book	A second patient row is added at the same time (slot_type double_booked)
Step 4 – reschedule	The low-priority patient is replaced by the emergency patient
Step 5 – overbook	A new slot is added after 16:30 (17:00, then 17:30, ...)
Step 0 / routine next day	No change to tomorrow's schedule; the request is only logged

Every confirmed booking is also recorded in an **emergency_bookings** table (time, patient, triage, priority, decision, affected patient), giving an audit trail. Safety checks prevent mistakes: a patient already booked with that doctor gets **ALREADY BOOKED** instead of a second slot, and a slot that has already been double-booked is not double-booked again. A reset option restores the original schedule before a demo.

```
python emergency_scheduler.py --patient P0098 --doctor D01 --triage 2 --book
BOOKING : Booked P0098 with D01 at 14:30.
python emergency_scheduler.py --patient P0024 --doctor D01 --triage 2 --book
DECISION : Step 4 - RESCHEDULE LOWEST-PRIORITY PATIENT (14:30 is now used)
BOOKING : Booked P0024 with D01 at 11:30. P0041 has been moved to the next
day.
```

10.8 How It Connects to the Pipeline

The ETL pipeline runs in batch (for example every night) and keeps the risk scores, no-show history and schedule up to date in MySQL. The scheduler reads these tables when an emergency request comes in and decides within seconds. The batch pipeline provides the history; the scheduler uses it for a live decision.

11. Interactive Web Application (app.py)

To make the solution usable by hospital staff without typing commands, a clickable web application was built with **Streamlit**, a Python library that turns a Python script into a web page. It reads and writes the same MySQL tables produced by the pipeline. It is started with one command, `streamlit run app.py`, and opens in the browser at `http://localhost:8501`.

Page	What the user can do
Dashboard	See key numbers (patients, high-risk count, average wait, no-show rate, tomorrow's bookings) and charts
Emergency Booking	Choose patient, doctor and triage level → click Find best slot → review the checks and decision → click Confirm booking
Doctor Schedule	Colour-coded grid of every doctor's slots for tomorrow; emergency bookings appear with an ambulance mark
Bookings Log	List of all emergency bookings; Reset schedule button
Run Pipeline	Generate sample data and run the full ETL pipeline with one click, with the log shown on screen



Figure 5: Dashboard page



Emergency Booking

Choose the patient, doctor and triage level, then click Find best slot. Review the decision and click Confirm booking to save it.

Patient

P0098 · Divya Varma · High risk (10)

Doctor

D01 · Cardiology · 14/14 booked

Triage level (set by the nurse)

- ☐ 1 – Immediate (life-threatening)
☒ 2 – Very urgent
☐ 3 – Urgent
☐ 4 – Routine

Divya Varma · 62 yrs · Female

Chronic condition: Heart Disease

Risk: High (score 10) · missed 1 of 5 past visits

Priority score = (5 – 2) × 10 + 10 = 40

Checks made (least disruptive option first)

✗ Emergency buffer slot free

✓ Likely no-show patient to double-book

Step 2: DOUBLE-BOOK LIKELY NO-SHOW SLOT

Doctor D01 at 14:30

P0034 missed 2 of 5 past visits (40%). Both are kept; confirm by phone.

Other patient affected: P0034

✓ Confirm booking

✗ Cancel

Figure 6: Emergency Booking page – each check is shown with ✓✗ before the decision

In Figure 6, the nurse selected patient P0098 (High risk), cardiologist D01 and triage level 2. The app shows the priority calculation, the checks it made, and the decision to double-book the 14:30 slot of a patient who often misses appointments. After **Confirm booking**, the Doctor Schedule page updates immediately (Figure 7).



Doctor Schedule – 2026-09-22

● free slot ● free emergency buffer ● booked 🚑 emergency booking

doctor_id	department	09:00	09:30	10:00	10:30	11:00	11:30	12:00	12:30	14:00	14:30	15:00
D01	Cardiology	● P0080	● P0157	● P0132	● P0071	● P0175	● P0041	● P0199	● P0125	● P0029	● P0034 + 🚑 P0098	● P00...
D02	Cardiology	● P0046	● P0140	● P0002	● P0126	● P0174	● P0010	● P0084	● P0017	● P0134	● P0106	● P00...
D03	General Medicine	● P0190	● P0189	● P0196	● buffer	● P0153	● free	● P0096	● P0127	● P0168	● P0012	● P01...
D04	General Medicine	● P0103	● P0155	● P0118	● buffer	● P0074	● P0088	● free	● P0107	● free	● P0173	● free
D05	General Medicine	● P0121	● P0079	● P0105	● buffer	● P0075	● P0011	● P0167	● P0166	● P0014	● P0038	● P01...
D06	Orthopedics	● P0068	● P0047	● P0085	● P0093	● P0135	● P0122	● P0027	● P0056	● P0116	● P0130	● P01...
D07	Orthopedics	● P0111	● free	● P0129	● buffer	● P0146	● P0194	● P0139	● P0055	● P0021	● P0040	● P00...
D08	Pediatrics	● P0138	● P0185	● P0037	● buffer	● P0112	● P0007	● P0165	● P0164	● P0195	● P0008	● P00...
D09	Neurology	● P0058	● P0128	● P0051	● P0072	● P0042	● P0179	● P0169	● P0187	● P0078	● P0094	● P00...
D10	Pulmonology	● P0009	● P0152	● P0158	● buffer	● P0115	● P0044	● free	● free	● P0181	● free	● P00...

Slot details

Doctor

D01

time	slot_type	slot_status	patient_id	name	risk_level	triage_level
09:00	regular	Booked	P0080	Bhavya Sharma	Medium	4

Figure 7: Doctor Schedule page after the booking – D01 at 14:30 shows the emergency patient

12. How to Run the Project

```
pip install -r requirements.txt
export DB_PASSWORD='your_mysql_password' # Windows: set DB_PASSWORD=...
python generate_data.py # create sample source data (once)
python pipeline.py # run ETL + print insights
```

```
python visualize.py          # create dashboard charts
python emergency_scheduler.py # emergency prioritisation demo
streamlit run app.py         # open the clickable web app

# terminal booking (without the app):
python emergency_scheduler.py --patient P0098 --doctor D01 --triage 2 --book
python emergency_scheduler.py --reset      # undo all bookings
```

The database hospital_db is created automatically. The tables can then be explored in MySQL Workbench, for example: `SELECT * FROM patient_risk_scores WHERE risk_level = 'High';`

13. Data Security and Governance

Hospital data is highly sensitive, so the design includes the following safeguards:

- **No hard-coded passwords:** credentials are read from environment variables, so they are never stored in code or uploaded to GitHub.
- **Validation gate:** incorrect data is stopped before it reaches the database.
- **Logging and audit trail:** every run is recorded in logs/pipeline.log.
- **Sample data only:** no real patient information is used.
- **In production:** role-based access (e.g. reception cannot see diagnoses), encryption, masking of names and phone numbers for analysts, and compliance with India's Digital Personal Data Protection (DPDP) Act 2023 and ABDM health data guidelines.

14. Challenges and Solutions

Challenge	Solution in this project
Different data formats (CSV, JSON, text)	Separate extract function per source; JSON flattened with json_normalize
Inconsistent values (gender, text case)	Standardisation mapping in transform step
Wearable sensor errors	Removed readings outside physically possible ranges defined in config.py
Invalid or orphan records	Referential integrity check against the patient list; invalid times removed
Bad data reaching the database	Validation step stops the pipeline on any failed check
Privacy of patient data	Sample data, environment-variable credentials, governance plan
Emergency patient when doctor is fully booked	Priority score (triage + risk score) and a 6-step slot-finding strategy in emergency_scheduler.py
Same patient booked twice by repeated clicks	Scheduler detects an existing booking and returns ALREADY BOOKED

15. Future Scope

The current project is a batch ETL pipeline. As data volume grows, it can be extended using the following concepts from Data Engineering Fundamentals:

- **Data lake (Bronze / Silver / Gold layers):** store raw files in cloud storage (Amazon S3 / Azure Data Lake). Our data/raw and data/processed folders are a small-scale version of the Bronze and Silver layers.
- **Data warehouse:** move analytical tables to Snowflake, BigQuery or Redshift and model them as a star schema (fact_appointments, fact_lab_results, fact_vitals with dim_patient, dim_doctor, dim_department, dim_date).
- **Real-time scheduling:** connect the booking app to the hospital's live appointment system, add staff login, and send automatic SMS to rescheduled patients.
- **Real-time streaming:** send wearable readings through Apache Kafka and process them with Spark Structured Streaming to raise instant alerts for dangerous vitals.
- **Orchestration:** schedule the pipeline daily with Apache Airflow instead of running it manually.
- **Big data processing:** replace Pandas with Apache Spark for millions of records.
- **Machine learning:** replace the rule-based score with a trained model (e.g. logistic regression or random forest) predicting readmission risk.
- **NLP:** use medical NLP models instead of keyword matching to extract diagnoses from doctor notes.
- **Dashboards:** connect Power BI or Tableau directly to MySQL for live dashboards.

16. Conclusion

This project implements a complete, working ETL pipeline for a multi-specialty hospital. It extracts data from five different source systems, cleans and enriches it, validates data quality, and loads seven analysis-ready tables into MySQL. The analysis identified the departments and hours with the longest waiting times, flagged 7 high-risk patients for early attention, and highlighted the most common health issues. The emergency scheduler and web application show how the same data supports live operational decisions, fitting urgent patients into fully booked schedules with the least disruption to others. The modular design means each stage can be scaled up with data lake, warehouse, streaming and machine learning technologies in future.